

- P4\_CPU设计文档——Verilog实现单周期处理器
  - 搭建CPU第一部分：CPU局部模块的设计
    - IFU
      - PC子模块
      - NPC子模块
      - IM子模块
      - 把三个小模块整合到一起
    - Splitter模块
    - GRF
    - ALU
    - DM
    - EXT
    - Controller
  - 搭建CPU第二部分：CPU顶层模块的设计
  - 搭建CPU第三部分：verilog源文件的编译和输出调试
- P4\_CPU设计文档-思考题
- 写在最后

## P4\_CPU设计文档——Verilog实现单周期处理器

既然设计文档也没有什么内容和格式上的要求，我就借此机会梳理总结一下在本周搭建的完整过程和搭建CPU时我的一些感想和体会。

本次实验的任务是使用Verilog语言写一个32位单周期处理器，需要支持的指令集为：`add`, `sub`, `ori`, `lw`, `sw`, `beq`, `lui`, `jal`, `jr`, `nop`，这在P3实验的基础上新增了一些指令，但是总体来说搭建CPU的思路是一致的。仍然先搭建不同的模块，最后把所有模块整合到顶层模块mips.v中。有了P3的基础上，CPU每个模块以及整体数据通路的具体架构就不再是P4需要重点关注的事情了，这一部分直接参考P3即可。P4需要考虑的事情是怎样用代码来实现这些功能。

# 搭建CPU第一部分：CPU局部模块的设计

我沿用P3的设计，整个CPU分为7个一级模块以及IFU调用的3个二级模块，它们分别是：

- IFU
  - PC
  - NPC
  - IM
- Splitter
- GRF
- ALU
- DM
- EXT
- Controller

需要注意的是P4要求使用同步复位信号reset，因此在所有时序逻辑中需要小心。下面具体来看每一个模块的搭建过程。

## IFU

IFU分为三个小模块，这里需要同时考虑 `beq``jal``jr` 三种跳转方式和正常顺序执行。因此NPC模块的计算方式在P3的基础上有增加。

### PC子模块

PC寄存器就是一个寄存器，里面存着当前指令的地址。唯一需要注意的就是初始地址不是0x0000\_0000而是0x00003000，输入输出端口如下：

| 信号名          | 方向 | 描述               |
|--------------|----|------------------|
| clk          | I  | 时钟信号             |
| reset        | I  | 同步复位信号           |
| nextPC[31:0] | I  | 下一条指令的地址，来自NPC模块 |

| 信号名             | 方向 | 描述                                      |
|-----------------|----|-----------------------------------------|
| currentPC[31:0] | O  | 当前指令的地址，发送给IM用于取出机器码，发送给NPC用于计算下一条指令的地址 |

代码实现如下：

```

8      reg [31:0] pc_register;
9
10     always@(posedge clk) begin
11         if (reset) begin
12             pc_register <= 32'b0;
13         end
14         else begin
15             pc_register <= nextPC - 32'h0000_3000;
16         end
17     end
18
19     always@(*) begin
20         currentPC = pc_register + 32'h0000_3000;
21     end

```

### NPC子模块

在考虑到J型指令和 jr 跳转指令后，NPC共有4中计算方式。同时要注意的是 jal 指令需要将  $PC + 8$  写入到 \$ra 寄存器。因此输入输出端口定义如下：

| 信号名             | 方向 | 描述                                                  |
|-----------------|----|-----------------------------------------------------|
| currentPC[31:0] | I  | 当前指令的地址                                             |
| NPC_op[1:0]     | I  | 指令计算操作码，00表示顺序执行，01表示 beq 跳转，10表示 jal 跳转，11表示 jr 跳转 |
| beq_zero        | I  | 来自ALU模块，高电平表示 beq 指令的跳转条件成立；低电平表示跳转指令不成立            |
| imm16[15:0]     | I  | 来自Splitter模块， beq 指令的16位立即数                         |
| imm26[25:0]     | I  | 来自Splitter模块， jal 指令的26位立即数                         |
| add32[31:0]     | I  | 来自GRF模块 \$ra 寄存器， j 指令的32位操作数                       |
| nextPC[31:0]    | O  | 计算好的下一条指令的地址                                        |

| 信号名                      | 方向 | 描述                                            |
|--------------------------|----|-----------------------------------------------|
| jal_return_address[31:0] | O  | $PC + 4$ 的值，需要保存到 $\$ra$ 寄存器中，作为调用函数结束以后返回的地址 |

这里有一个需要格外注意的点！根据mips指令集，实际上 **jal** 指令保存到  $\$ra$  寄存器的数值应该是  $PC + 8$ ！这是因为我们现在搭建的是**不考虑延迟槽的单周期处理器**！所以在这里保存  $PC + 4$ ，也就是下一条指令作为函数调用结束返回后执行的指令。

四种计算方式的宏定义为：

```
1  `define NPC_SEQ 2'b00
2  `define NPC_BEQ 2'b01
3  `define NPC_JAL 2'b10
4  `define NPC_JR 2'b11
```

具体的计算过程代码实现为：

```
19  case(NPC_op)
20      `NPC_SEQ: begin
21          nextPC = currentPC + 32'h0000_0004;
22      end
23      `NPC_BEQ: begin
24          if (beq_zero) begin
25              nextPC = currentPC + 32'h0000_0004 + {{14{imm16[15]}}},
26          end
27          else begin
28              nextPC = currentPC + 32'h0000_0004;
29          end
30      end
31      `NPC_JAL: begin
32          nextPC = {currentPC[31:28], imm26, 2'b00};
33      end
34      `NPC_JR: begin
35          nextPC = add32;
36      end
37  endcase
```

## IM子模块

在Logisim中，可以直接使用内置的RAM和ROM模块进行仿真。而在Verilog代码中则没有这样的功能，需要我们用庞大的寄存器阵列来模拟内存。而Verilog提供了读写寄存器的内置函数，因此将程序的若干条机器码导入ROM的工作可以直接使用内置函数  $\$readmemh$  来完成。输入输出端口定义如下：

| 信号名               | 方向 | 描述        |
|-------------------|----|-----------|
| currentPC[31:0]   | I  | 当前指令的地址   |
| instruction[31:0] | O  | 取出的32位机器码 |

这里IM的容量为 $4096 * 32\text{bit}$ ，即4096个32位寄存器。指令的地址是从0x00003000开始的，而我们的指令保存地址是从0x00000000开始的，因此取指令时需要给输入的地址减去0x00003000。

指令的地址范围是从0x00003000~0x00006FFF，而且每条指令占四字节，所以实际上ROM的地址只需要12位就满足要求，因此我们需要取每个地址的第2位到第13位作为ROM取数据的参考。详细的解释参考下面的DM模块(我先写的DM，后写的IM)。

具体的代码实现如下：

```

6      reg [31:0] memory[4095:0];
7      wire [31:0] IM_address = currentPC - 32'h0000_3000;
8
9      initial begin
10         $readmemh("code.txt",memory);
11     end
12
13     assign instruction = memory[ IM_address[13:2] ];

```

### 把三个小模块整合到一起

我们需要在IFU.v模块里实例化这三个子模块，然后按照逻辑搭建出大的IFU模块。首先IFU的输入输出端口定义如下：

| 信号名         | 方向 | 描述                                  |
|-------------|----|-------------------------------------|
| clk         | I  | 时钟信号                                |
| reset       | I  | 同步复位信号                              |
| NPC_op[1:0] | I  | 来自控制模块的信号，表示下一条指令地址的计算方式            |
| beq_zero    | I  | 来自ALU模块的信号，表示 beq 指令的两个目标寄存器的值是否相等  |
| imm16[15:0] | I  | 来自Splitter模块， beq 指令的立即数，表示跳转的地址偏移量 |

| 信号名                      | 方向 | 描述                                             |
|--------------------------|----|------------------------------------------------|
| imm26[25:0]              | I  | 来自Splitter模块，jal指令的立即数，表示跳转的地址偏移量              |
| add32[31:0]              | I  | 来自GRF模块 \$ra寄存器，j指令的32位操作数                     |
| currentPC[31:0]          | O  | 输出当前指令的地址(这是本次实验的要求，其实在CPU的工作中不需要将这个信号输出到模块外部) |
| instruction[31:0]        | O  | 输出当前的指令(这是这个模块最关键的输出信号! )                      |
| jal_return_address[31:0] | O  | $PC + 8$ 的值，需要保存到 \$ra寄存器中，作为调用函数结束以后返回的地址     |

首先明确IFU这个大模块的输入输出端口定义：

```

1  module IFU(
2      input clk,
3      input reset,
4      input [1:0] NPC_op,
5      input beq_zero,
6      input [15:0] imm16,
7      input [25:0] imm26,
8      input [31:0] add32,
9      output [31:0] currentPC,
10     output [31:0] instruction,
11     output [31:0] jal_return_address
12 );

```

然后声明了需要的中间变量后将三个子模块实例化：

```
14     wire [31:0] nextPC;
15
16     PC pc(
17         .clk(clk),
18         .reset(reset),
19         .nextPC(nextPC),
20         .currentPC(currentPC)
21     );
22     NPC npc(
23         .currentPC(currentPC),
24         .NPC_op(NPC_op),
25         .beq_zero(beq_zero),
26         .imm16(imm16),
27         .imm26(imm26),
28         .add32(add32),
29         .nextPC(nextPC),
30         .jal_return_address(jal_return_address)
31     );
32     IM im(
33         .currentPC(currentPC),
34         .instruction(instruction)
35     );
```

# Splitter模块

综合考虑所有需要支持的指令以及它们不同的类型，我们需要把一个32位机器码划分成：

- R型指令：opcode[5:0], rs[4:0], rt[4:0], rd[4:0], shamt[4:0], funct[5:0]，共6段；
- J型指令：
- I型指令：opcode[5:0], rs[4:0], rt[4:0], imm16[15:0]，共4段。

因此Splitter模块就是要根据32位机器码给出这些字段，故它的输入输出端口定义如下：

| 信号名               | 方向 | 描述                                                |
|-------------------|----|---------------------------------------------------|
| instruction[31:0] | I  | 当前指令的32位机器码                                       |
| opcode[5:0]       | O  | 一个指令的操作码，发送给控制模块用于识别不同的指令                         |
| rs[4:0]           | O  | rs寄存器的编号(针对I型指令和R型指令)                             |
| rt[4:0]           | O  | rt寄存器的编号(针对I型指令和R型指令)                             |
| rd[4:0]           | O  | rd寄存器的编号(针对R型指令)                                  |
| shamt[4:0]        | O  | 移位量，对于非移位指令这一段为0，不需要关注(针对R型指令)                    |
| funct[5:0]        | O  | 功能码，用于明确一条R型指令的具体功能，也就是说和opcode共同区分一条R型指令(针对R型指令) |
| imm16[15:0]       | O  | 16位立即数(针对I型指令)                                    |
| imm26[25:0]       | O  | 26位地址(针对J型指令)                                     |

在代码的编写过程中难度不大，熟练掌握数据的位拼接的语法即可。具体的代码实现如下：



```
1  module Splitter(  
2      input [31:0] instruction,  
3      output [5:0] opcode,  
4      output [4:0] rs,  
5      output [4:0] rt,  
6      output [4:0] rd,  
7      output [4:0] shamt,  
8      output [5:0] funct,  
9      output [15:0] imm16,  
10     output [25:0] imm26  
11 );  
12  
13     assign opcode = instruction[31:26];  
14     assign rs = instruction[25:21];  
15     assign rt = instruction[20:16];  
16     assign rd = instruction[15:11];  
17     assign shamt = instruction[10:6];  
18     assign funct = instruction[5:0];  
19     assign imm16 = instruction[15:0];  
20     assign imm26 = instruction[25:0];  
21  
22 endmodule
```

# GRF

这个模块是32个32位寄存器组成的寄存器堆，使用数组即可。注意这里需要使用同步复位信号，并且要保持0号寄存器的值始终为零。它的写使能信号来自控制器。它的输入输出端口定义如下：

| 信号名       | 方向 | 描述                 |
|-----------|----|--------------------|
| clk       | I  | 时钟信号               |
| reset     | I  | <b>同步复位</b> 信号     |
| GRF_WE    | I  | 写使能信号，高电平时可以写入数据   |
| A1[4:0]   | I  | rs寄存器编号(第一个读数据的端口) |
| A2[4:0]   | I  | rt寄存器编号(第二个读数据的端口) |
| A3[4:0]   | I  | rd寄存器编号(写数据的端口)    |
| WD[31:0]  | I  | 将要写入rd寄存器的数据       |
| RD1[31:0] | O  | rs寄存器中保存的数据        |
| RD2[31:0] | O  | rt寄存器中保存的数据        |

在写代码的时候，这里有一个需要注意的点是写数据是只有在时钟上升沿，也就是说写入数据是时序逻辑。而读数据需要按照组合逻辑来赋值。

注意组合逻辑用阻塞赋值，时序逻辑用非阻塞赋值。

代码分成模块声明，同步复位和写数据(时序逻辑)，读数据(组合逻辑)三块，具体的实现如下：

- 模块声明：

```

1  module GRF(
2      input [4:0] A1,
3      input [4:0] A2,
4      input [4:0] A3,
5      input [31:0] WD,
6      input WE,
7      input clk,
8      input reset,
9      output reg [31:0] RD1,
10     output reg [31:0] RD2
11 );

```

- 时序逻辑

```

16     always@(posedge clk) begin
17         if (reset) begin
18             for (i = 0; i < 32; i = i + 1) begin
19                 register[i] <= 32'b0;
20             end
21         end
22         else begin
23             if (WE && (A3 != 5'b00)) begin
24                 register[A3] <= WD;
25             end
26             else begin
27                 register[A3] <= register[A3];
28             end
29         end
30     end

```

- 组合逻辑

```

32     always@(*) begin
33         RD1 = (A1 == 5'b0) ? 32'b0 : register[A1];
34         RD2 = (A2 == 5'b0) ? 32'b0 : register[A2];
35     end

```

# ALU

就目前的指令集来看，32位运算器需要支持的运算有加减法，逻辑位或，逻辑左移16位四种。另外，ALU还需要能够进行两个数是否相等的比较，供 `beq` 指令进行判断。它的计算方式信号来自控制器。这个模块是组合逻辑，因此不涉及时钟和复位信号。它具体的输入输出端口定义如下：

| 信号名         | 方向 | 描述                                               |
|-------------|----|--------------------------------------------------|
| A[31:0]     | I  | rs寄存器中存储的数据                                      |
| B[31:0]     | I  | rt寄存器中存储的数据                                      |
| ALU_op[1:0] | I  | 算路逻辑单元的操作码，00表示加法，01表示减法，10表示或逻辑运算，11表示向左逻辑位移16位 |
| ans[31:0]   | O  | 32位计算结果，发送到GRF就保存在寄存器中，发送到IM就保存到内存中              |
| zero        | O  | 判断两数是否相等，发送到NPC作为 <code>beq</code> 是否跳转的信号       |

不同的计算模式使用宏定义即可。

使用case语句块注意写default分支。

以下是具体的代码实现：这个模块现在支持的四种计算方式为：

```
1  `define ALU_ADD 2'b00
2  `define ALU_SUB 2'b01
3  `define ALU_ORI 2'b10
4  `define ALU_LUI 2'b11
```

每种计算的具体实现和为 `beq` 指令实现做的比较为：

```
13     always@(*) begin
14         zero = (A == B)? 1'b1 : 1'b0;
15     end
16
17     always@(*) begin
18         case(ALU_op)
19             `ALU_ADD: ans = A + B;
20             `ALU_SUB: ans = A - B;
21             `ALU_ORI: ans = A | B;
22             `ALU_LUI: ans = {B[15:0], 16'b0};
23             default: ans = 32'b0;
24         endcase
25     end
```

如需扩展这个模块，只需要增加 `ALU_op` 信号的位宽以增加更多的计算模式即可。这一点要比在Logisim中修改多路选择器要方便的多。

# DM

Logisim中DM模块使用RAM来进行模拟，此处我们只能用寄存器阵列来模拟，用Verilog的内置函数 `$display` 来输出其中的数据。这里DM的容量为  $3072 * 32\textit{bit}$  即3072个32位寄存器。同时DM需要接收控制器给它的写使能信号才能进行写操作。具体的输入输出端口定义如下：

| 信号名               | 方向 | 描述                           |
|-------------------|----|------------------------------|
| clk               | I  | 时钟信号                         |
| reset             | I  | <b>同步复位</b> 信号               |
| DM_WE             | I  | 写使能信号，高电平时写入数据，低电平时读出目标地址的数据 |
| address[31:0]     | I  | 写数据的地址                       |
| data_input[31:0]  | I  | 写入的数据                        |
| data_output[31:0] | O  | 读出的数据                        |

这里最重要的问题就是怎样把32位读写数据的地址和IM内部存储的地址对应起来。根据IM的容量可知其内部存了不超过  $2^{12}$  个字，每个字对应一个寄存器。也就是说32位地址是有上限的。同时由于这里我们都是按字取值，因此32位地址一定是经过对齐的，否则就是无效的指令。因此32位地址的低2位一定是0，于是有效的地址就是原地址的第13到第2位。

综上，DM的真实地址只截取 `address[13:2]` 即可。

如果出现按字节寻址的指令，我们在进行扩展即可。

由于这里读写数据共用一个地址，因此在时钟上升沿且写使能信号有效时进行写操作。写使能信号无效时进行读操作。具体的代码实现如下：

```
14  always@(posedge clk) begin
15      if (reset) begin
16          for (i = 0; i < 3072; i = i + 1) begin
17              memory[i] <= 32'b0;
18          end
19      end
20      else begin
21          if (DM_WE) begin
22              memory[address_real] <= data_input;
23          end
24      end
25  end
26
27  always@(*) begin
28      if (!DM_WE) begin
29          data_output = memory[address_real];
30      end
31  end
```

# EXT

由于EXT模块是为ALU的计算来处理立即数，考虑到ALU的计算模式，EXT只需要能够进行16位立即数的零扩展和符号扩展即可。关于J型指令的26位立即数的扩展则内置到NPC模块里独立计算即可。这样可以降低控制信号的复杂度。因此我在P3的基础上把关于26位立即数的处理在这个模块删除了。具体输入输出端口定义如下：

| 信号名           | 方向 | 描述                                             |
|---------------|----|------------------------------------------------|
| imm16[15:0]   | I  | 16位立即数                                         |
| EXT_op        | I  | 来自控制单元的控制信号，低电平表示16位立即数进行零扩展，高电平表示16位立即数进行符号扩展 |
| imm_ext[31:0] | O  | 32位扩展结果，发送到ALU进行下一步的计算                         |

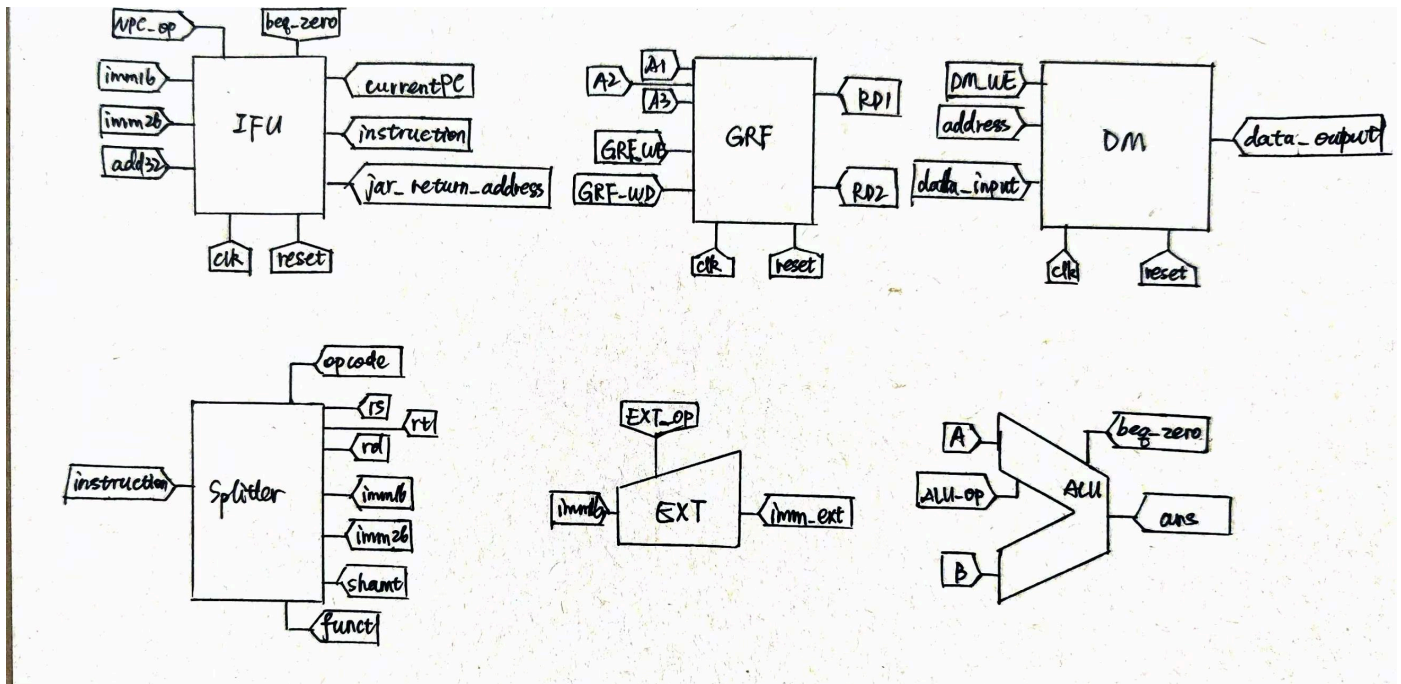
一共两种扩展模式，具体的代码实现如下：

```
7      always@(*) begin
8          if (EXT_op == 1) begin
9              imm32 = {{16{imm16[15]}}, imm16};
10         end
11         else begin
12             imm32 = {16'b0, imm16};
13         end
14     end
```

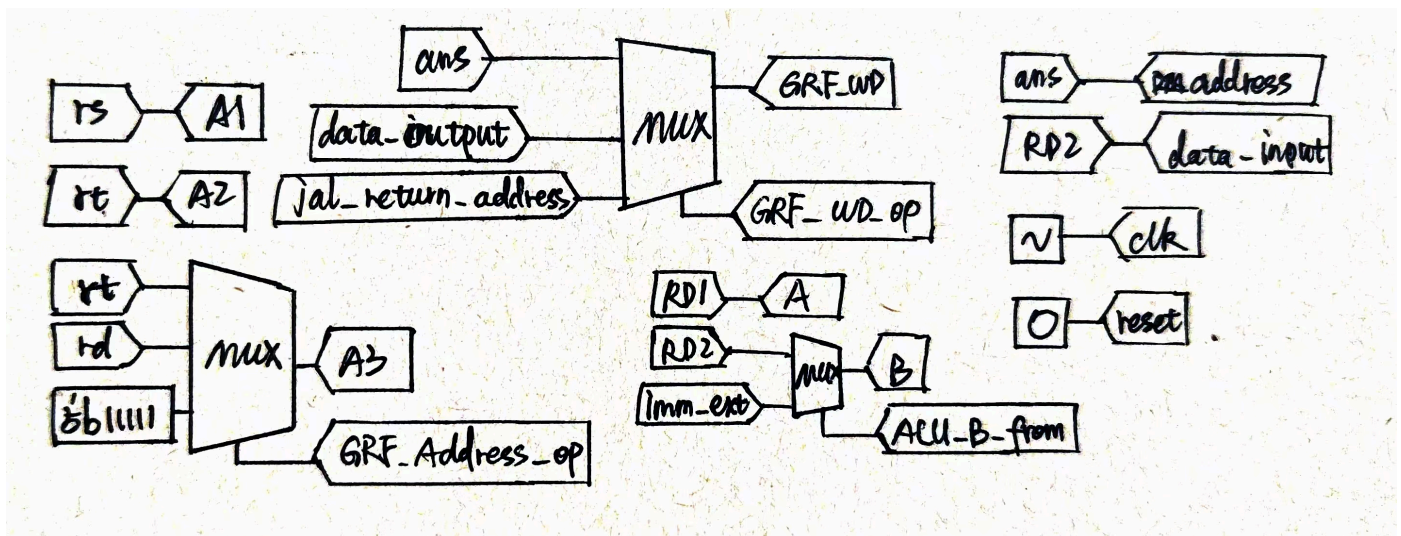


# Controller

到搭建控制器之前，其他所有模块就应该都搭建好了。六个大模块的输入输出示意图如下：



目前各个模块的写使能信号有两个：`GRF_WE`和`DM_WE`；各个模块的操作模式有三个：`NPC_op[1:0]`和`EXT_op`和`ALU_op`；再加上构建数据通路用到的选择信号(即多路选择器的选择信号)，如下图所示，共有：



(1). `GRF_Address_from[1:0]`: 写入GRF的地址是 `$rt`寄存器还是 `$rd`寄存器抑或是 `jal`指令独用的常数地址 `5'b11111`寄存器；

经过分析，R型指令写入 `$rd`寄存器，需要回写的I型指令以及 `jal`指令写入 `$rt`寄存器。

(2). `GRF_WD_from[1:0]`: 写入GRF的数据来自于ALU的计算结果 `ans` 还是DM取出的数据 `data_output`抑或是 `jal`指令独有的 `jal_return_address`。

只有 `lw`指令写入的数据是来自 `data_output`；只有 `jal`指令的数据来自于 `jal_return_address`

(3). `ALU_B_from`: ALU的第二个操作数来自于 `$rt`寄存器还是EXT输出的32位立即数 `imm_ext`。

显然只有R型的计算指令来自于 `$rt`寄存器读取的数据 `RD1`

因此我们共有8个控制信号需要由控制器给出，控制器的输入输出端口定义如下：

| 信号名                                | 方向 | 描述                                                                                                         |
|------------------------------------|----|------------------------------------------------------------------------------------------------------------|
| <code>opcode[5:0]</code>           | I  | 不同指令的操作码                                                                                                   |
| <code>funct[5:0]</code>            | I  | R型指令辅助操作码对指令进行区分的字段                                                                                        |
| <code>NPC_op[1:0]</code>           | O  | 发送给IFU的NPC模块，00表示正常顺序执行，01表示识别到 <code>beq</code> 指令，10表示识别到 <code>jal</code> 指令，11表示识别到 <code>jr</code> 指令 |
| <code>GRF_WE</code>                | O  | GRF的写使能信号，高电平有效                                                                                            |
| <code>GRF_Address_from[1:0]</code> | O  | 发送到GRF写数据地址端口的 <b>选择信号</b> ，00时写入数据的地址为rt寄存器,01时写入数据的地址为rd寄存器，10时写入数据的地址是5' <code>b11111</code> 寄存器        |
| <code>GRF_WD_from[1:0]</code>      | O  | 发送到GRF写数据数据端口的 <b>选择信号</b> ，00时写入ALU的计算结果,01时写入从内存中取出的数据，10时写入 <code>jal_return_address</code>             |
| <code>ALU_op[1:0]</code>           | O  | 发送到ALU的操作码。00表示加法，01表示减法，10表示逻辑或立即数，11表示逻辑左移16位                                                            |
| <code>ALU_B_from</code>            | O  | 发送到ALU第二个操作数的 <b>选择信号</b> ，高电平时将立即数扩展后的结果输入到ALU的第二操作数，低电平时将GRF取出的RD2写入ALU的第二操作数                            |
| <code>DM_WE</code>                 | O  | DM的写使能信号，高电平有效                                                                                             |
| <code>EXT_op</code>                | O  | 发送到EXT的操作码，高电平时进行符号扩展，低电平时进行零扩展                                                                            |

我们根据识别出不同的指令，给出每条指令对应的每个指令。这样在添加指令的时候可以有很好的扩展性。因此我根据不同的 `opcode` 和 `funct` 字段先识别出不同的指令，每条指令对应一个宏定义，如下：

```
1  `define R 6'b000000
2  `define R_ADD 6'b100000
3  `define R_SUB 6'b100010
4  `define R_JR 6'b001000
5  `define ORI 6'b001101
6  `define LW 6'b100011
7  `define SW 6'b101011
8  `define BEQ 6'b000100
9  `define LUI 6'b001111
10 `define JAL 6'b000011
```

然后根据不同指令的数据通路给出每条指令的控制信号，具体代码如下：

```
24  always@(*) begin
25      case(opcode)
26          `R:begin
27              case(funct)
28 >                  `R_ADD:begin ...
38 >                  `R_SUB:begin ...
48 >                  `R_JR:begin ...
58 >                  default: begin ...
68              endcase
69          end
70 >          `ORI: begin ...
80 >          `LW: begin ...
90 >          `SW: begin ...
100 >          `BEQ: begin ...
110 >          `LUI: begin ...
120 >          `JAL: begin ...
130 >          default: begin ...
140      endcase
141  end
```

到这里我们就给出了每一条指令的数据通路和控制信号。

# 搭建CPU第二部分：CPU顶层模块的设计

有了每一个小的模块，我们就可以在顶层模块中实例化这些小模块，组成一个完整的CPU电路。在顶层模块中，我们只有两个输入端口，没有输出端口，定义如下：

| 信号名   | 方向 | 描述     |
|-------|----|--------|
| clk   | I  | 时钟信号   |
| reset | I  | 异步复位信号 |

根据实验的要求，我们需要随时监控GRF和DM的状态，并在这两个模块有变化的时候给出相应的输出。

- 监控GRF模块：我们需要输出的信息是：当时钟上升沿到来时，若写使能信号有效，则输出

```
$display("@%h: %d <= %h", currentPC, A3, GRF_WD);
```

- 监控DM模块：我们需要输出的信息是：当时钟上升沿到来时，若写使能信号有效，则输出

```
$display("@%h: *%h <= %h", currentPC, address, data_input);
```

因此，我们需要将 `currentPC` 信号输入到GRF和DM模块中，要注意在实际的处理器运行过程中不需要这么做。这里对这两个模块有一点小改动，因此特别说明一下。

在搭建顶层模块时，一个需要格外注意的工作是声明中间变量，以连接不同的子模块。在实际搭建过程中，我采取了逐个模块声明的方式，在每实例化一个模块之前声明这个模块需要但是未生明的信号。

要注意变量的位宽和类型。

具体的代码实现如下：

```

6      // Controller wires
7      wire [1:0] NPC_op;
8      wire GRF_WE;
9      wire [1:0] GRF_Address_from;
10     wire [1:0] GRF_WD_from;
11     wire [1:0] ALU_op;
12     wire ALU_B_from;
13     wire DM_WE;
14     wire EXT_op;
15
16     // IFU wires
17     wire [31:0] currentPC;
18     wire [31:0] instruction;
19     wire [31:0] jal_return_address;
20
21     // Splitter wires
22     wire [5:0] opcode;
23     wire [4:0] rs;
24     wire [4:0] rt;
25     wire [4:0] rd;
26     wire [4:0] shamt;
27     wire [5:0] funct;
28     wire [15:0] imm16;
29     wire [25:0] imm26;
30
31
32     wire beq_zero;
33     wire [31:0] add32;
34
35     assign add32 = RD1;

```

```

76     // GRF wires
77     wire [4:0] A1;
78     wire [4:0] A2;
79     wire [4:0] A3;
80     wire [31:0] WD;
81     wire [31:0] RD1;
82     wire [31:0] RD2;
83
84     assign A1 = rs;
85     assign A2 = rt;
86     assign A3 = (GRF_Address_from == 2'b00) ? rt :
87                 (GRF_Address_from == 2'b01) ? rd :
88                 (GRF_Address_from == 2'b10) ? 5'b111111 :
89                 5'b000000;
90     assign WD = (GRF_WD_from == 2'b00) ? ans :
91                 (GRF_WD_from == 2'b01) ? data_output :
92                 (GRF_WD_from == 2'b10) ? jal_return_address :
93                 32'b0;

```

|                                                                                                                                                                                                                                                            |                                                                                                                                                                                                     |
|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <pre> 117    // ALU wires 118    wire [31:0] A; 119    wire [31:0] B; 120    wire [31:0] ans; 121 122    assign A = RD1; 123    assign B = (ALU_B_from == 1'b0) ? RD2 : 124                (ALU_B_from == 1'b1) ? imm32 : 125                32'b0; </pre> | <pre> 134    // DM wires 135    wire [31:0] data_input; 136    wire [31:0] address; 137    wire [31:0] data_output; 138 139 140    assign data_input = RD2; 141    assign address = ans; 142 </pre> |
|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|

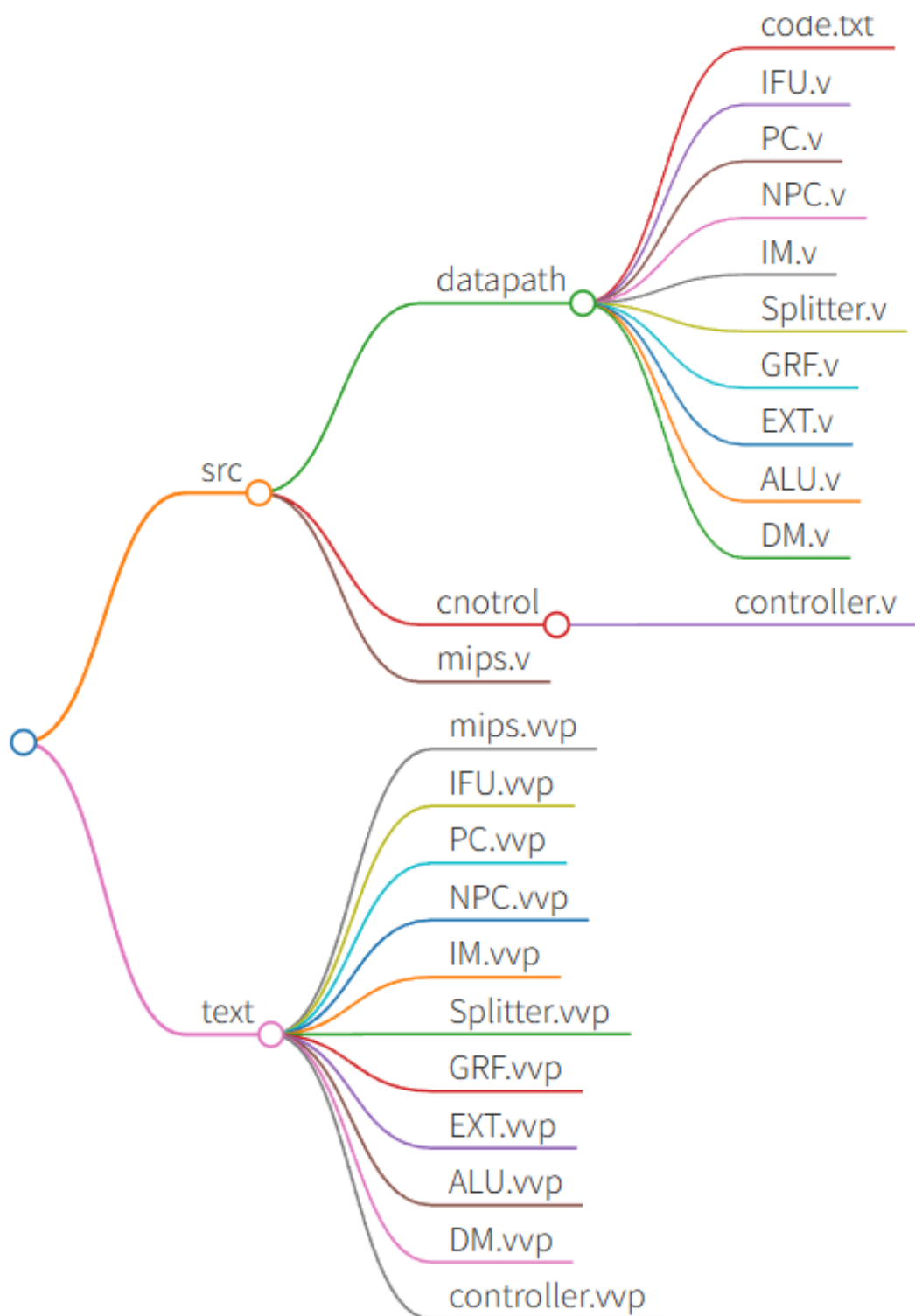
声明中间变量的过程，实际上就是连接不同模块构建数据通路的过程。然后依次实例化每个模块即可，具体代码这里就不展示了。实例化的语法可以参考IFU模块实例化PC和NPC和IM三个模块的代码。到这里整个CPU就构建完成了。VScode的TerosHDL插件给出了顶层模块的一个Module Documentation，我将其源文件和pdf格式的文件插入到下面，作为完整mips.v模块的一个展示。

[Module Documentation of mips.v](#)

[Module Documentation in pdf of mips.v](#)

# 搭建CPU第三部分：verilog源文件的编译和输出调试

以下是我的项目结构：



在完成实验的过程中，我是用命令行来进行verilog源文件的编译工作的。

(1) 编译独立的模块时直接编译即可，以PC为例：

```
iverilog -o test/PC.vvp src/PC.v
```

(2) 编译实例化其他模块的大模块时，需要同时将引用到的小模块编译进来，否则编译器找不到这个大模块中实例化的小模块，以IFU和mips为例：

```
iverilog -o test/IFU.vvp src/IFU.v src/PC.v src/NPC.v src/IM.v  
ivverilog -o test/mips.vvp src/mips.v src/datapath/*.v src/control/*.v
```

在编译完成后，我们可以试着运行这个vvd文件。当我调用顶层模块时：

```
vvd test/mips.vvd
```

可以看到我添加的调试代码相应的输出。这就表明搭建的这个模块可以正常运行。



# P4\_CPU设计文档-思考题

1. 阅读下面给出的DM的输入示例中(示例DM容量为4KB，即32bit x 1024字)，根据你的理解回答，这个 `addr` 信号又是从哪里来的？地址信号 `addr` 位数为什么是[11:2]而不是[9:0]？

| 文件   | 模块接口定义                                                                                                                                                                                                                                                                                                     |
|------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| dm.v | <pre>dm(clk, reset, MemWrite, addr, din, dout);<br/>    input  clk;    //clock<br/>    input  reset;  //reset<br/>    input  MemWrite; //memory write enable<br/>    input [11:2] addr; //memory's address for write<br/>    input [31:0] din;  //write data<br/>    output [31:0] dout; //read data</pre> |

问题一：在目前需要实现的指令集中，涉及到DM模块的只有 `sw` 和 `lw` 两条指令。它们的数据通路很相似，一个是向DM中写数据，一个是从DM中取数据。根据这两条指令的数据通路，输入DM的地址信号显然只能来自于ALU计算的结果(基地址加上偏移量)，这个计算结果是32位的。根据**DM的数据容量**，只需要  $10 * 2$  位二进制数据就能覆盖整个DM(示例的DM容量和实验的DM模块的容量不一样)的每一个字节，而且这两条指令都是以**字为单位**进行读写操作，因此根据**对齐**的原则有效地址的低两位一定都是零，否则无法取到一个完整的字。因此实际上有效的地址信号是不需要低两位的数据的。

2. 思考上述两种控制器设计的译码方式，给出代码示例，并尝试对比各方式的优劣。

问题二：两种方式分别是：(1)给出每条指令对应的8个控制信号；(2)给出每个控制信号在每条指令下的值。我认为就目前的工作量来看这两种方式还没有明显的优劣之分。但是个人认为使用方式一能够更加清晰地看出每条指令的数据走向，在添加指令的过程中也更为整顿。从代码的角度来看方式二的代码更加简洁明了，没有方式一那么繁琐。出于对添加指令的便捷程度的考虑，我采取了方式一进行开发。以下是两种方式的对比代码：

3. 在相应的部件中，复位信号的设计都是**同步复位**，这与 P3 中的设计要求不同。请对比**同步复位**与**异步复位**这两种方式的 `reset` 信号与 `clk` 信号优先级的关系。

问题三：P3中使用的是异步复位，这样的方式显然 `reset` 信号的优先级要高于 `clk`。无论现在是在哪一个时钟周期的哪一个阶段，只要复位信号有效就立刻复位。P4中使用的

是同步复位信号，这时 `clk` 的优先级要高于 `reset` 信号。同步复位的实现方式其实是**当复位信号有效时，在时钟上升沿将初始值或复位值赋值给寄存器。**

4. C 语言是一种弱类型程序设计语言。C 语言中不对计算结果溢出进行处理，这意味着 C 语言要求程序员必须很清楚计算结果是否会导致溢出。因此，如果仅仅支持 C 语言，MIPS 指令的所有计算指令均可以忽略溢出。请说明为什么在忽略溢出的前提下，`addi` 与 `addiu` 是等价的，`add` 与 `addu` 是等价的。提示：阅读《MIPS32® Architecture For Programmers Volume II: The MIPS32® Instruction Set》中相关指令的 Operation 部分。

问题四：不考虑溢出的情况下，`add` 和 `addu` 两条指令执行同样的工作。只是 `addu` 会在溢出的情况下给出一个异常。

# 写在最后

---